

MySQL Performance Benchmarking

1. Optimization Techniques

Before touching configurations, ensure your database logic is sound.

- **Indexing Strategy:**
 - **The Rule of Three:** Index columns used in WHERE, JOIN, and ORDER BY clauses.
 - **Composite Indexes:** For queries filtering by multiple columns (e.g., WHERE city='NY' AND status='active'), a single composite index on (city, status) is much faster than two separate ones.
 - **Avoid Over-Indexing:** Every index slows down INSERT and UPDATE operations.
- **Query Refactoring:**
 - **Never use SELECT *:** Only fetch the columns you need to reduce I/O and memory usage.
 - **Use EXPLAIN:** Prefix any slow query with EXPLAIN to see if it's performing a "Full Table Scan" or using an index.
- **Schema Design:**
 - **Smallest Data Types:** Use TINYINT instead of INT if the range allows. Use VARCHAR(50) instead of TEXT for short strings.

2. Key MySQL Configurations (my.cnf)

Parameter	Recommended Setting	Why it Matters
innodb_buffer_pool_size	70-80% of total RAM	The "cache" for your data and indexes. This is the #1 performance booster.
innodb_log_file_size	25% of buffer pool	Determines how much "redo" data can be stored before flushing to disk.
innodb_flush_method	O_DIRECT	Avoids "double buffering" by letting MySQL bypass the OS cache.
max_connections	151 (Default) to 500	Don't set this too high (e.g., 5000) or you'll exhaust memory via overhead.
query_cache_type	0 (Off)	In MySQL 8.0+, the query cache is removed/deprecated because it causes bottlenecks in high-concurrency.
innodb_file_per_table	ON	Ensures each table has its own .ibd file, making management and reclaiming space easier.

3. How to Benchmark Performance

Never make changes in production without testing. Benchmarking helps you find the "breaking point" of your database.

Tools to Use:

1. **Sysbench:** The industry standard for stress testing. It can simulate thousands of transactions per second (TPS).
 - *Command example:* `sysbench oltp_read_write --db-driver=mysql prepare`
2. **MySQLTuner:** A Perl script that analyzes your running instance and gives specific configuration recommendations.
 - *Usage:* Run `perl mysqltuner.pl` and look for the "Recommendations" section at the bottom.
3. **Slow Query Log:** Enable this to catch queries that take longer than seconds.

```
SET GLOBAL slow_query_log = 'ON';  
SET GLOBAL long_query_time = 1; -- Log anything over 1 second
```

Using Sysbench and MySQLTuner

Sysbench (The Stress Test)

Sysbench creates a fake database, runs thousands of queries, and reports how fast your server responded.

1. Install Sysbench

```
sudo apt update  
sudo apt install sysbench -y
```

2. Prepare the Test Database

Before testing, you need to create a dedicated database and fill it with dummy data. Log into MySQL and run:

```
CREATE DATABASE sbtest;  
CREATE USER 'sbusers'@'localhost' IDENTIFIED BY 'password123';  
GRANT ALL PRIVILEGES ON sbtest.* TO 'sbusers'@'localhost';
```

Then, tell Sysbench to "prepare" the data (this creates 10 tables with 100,000 rows each):

```
sysbench oltp_read_write --db-driver=mysql --mysql-db=sbtest --mysql-user=sbusers --mysql-  
password=password123 --tables=10 --table-size=100000 prepare
```

3. Run the Benchmark

Now, simulate a real load. This command runs a "Read/Write" test for 60 seconds using 16 simultaneous threads:

```
sysbench oltp_read_write --db-driver=mysql --mysql-db=sbtest --mysql-user=sbusers --mysql-  
password=password123 --tables=10 --table-size=100000 --threads=16 --time=60 run
```

4. Interpret the Results

Look for these two metrics in the output:

- **TPS (Transactions Per Second):** Higher is better. This is your raw throughput.
- **95th Percentile Latency:** Lower is better. This tells you that 95% of your queries finished faster than \$X\$ milliseconds.

MySQLTuner (The Diagnostic)

MySQLTuner analyzes your current settings and actual usage data to give you "Pro-Tips."

Crucial Tip: Only run MySQLTuner after your database has been running for at least **24 to 48 hours**. If you run it immediately after a restart, the data will be inaccurate.

1. Download the Script

```
wget http://mysqltuner.pl/ -O mysqltuner.pl  
chmod +x mysqltuner.pl
```

2. Run the Analysis

Run the script and provide your admin credentials when prompted:

```
./mysqltuner.pl
```

3. Read the Recommendations

The script will output a long report. Scroll to the very bottom to the "**Recommendations**" section. It will look like this:

- [!!] Maximum possible memory usage: 18G (110% of installed RAM) → **Warning:** You are over-allocating RAM; the server might crash.
- Variables to adjust: innodb_buffer_pool_size (>= 8G) → **Advice:** Increase this setting to improve speed.